

Unit 2 LINEAR DATA STRUCTURE

Syllabus	Array: Representation of arrays, Applications of arrays, sparse matrix and its representation Stack: Stack-Definitions & Concepts, Operations On Stacks, Applications of Stacks, Polish Expression, Reverse Polish Expression And Their Compilation, Recursion, Tower of Hanoi Queue: Representation Of Queue, Operations On Queue, Circular Queue, Priority Queue, Array representation of Priority Queue, Double Ended Queue, Applications of Queue Linked List: Singly Linked List, Doubly Linked list, Circular linked list, Linked implementation of Stack, Linked implementation of Queue, Applications of linked list.
-----------------	---

Linked List

➤ Linked Lists versus Arrays

- Both arrays and linked lists are a linear collection of data elements.
- But unlike an array, a linked list does not store its nodes in consecutive memory locations.
- Another point of difference between an array and a linked list is that a linked list does not allow random access of data.
- Nodes in a linked list can be accessed only in a sequential manner.
- But like an array, insertions and deletions can be done at any point in the list in a constant time.
- Another advantage of a linked list over an array is that we can add any number of elements in the list. This is not possible in case of an array.
- For example, if we declare an array as `int marks[20]`, then the array can store a maximum of 20 data elements only. There is no such restriction in case of a linked list.
- Linked lists provide an efficient way of storing related data and performing basic operations such as insertion, deletion, and updation of information at the cost of extra space required for storing the address of next nodes.

➤ Operation on Link List:

- Insert
 1. Insert at first position
 2. Insert at last position
 3. Insert into ordered list
- Delete
- Traverse list (Print list)

- Copy linked list

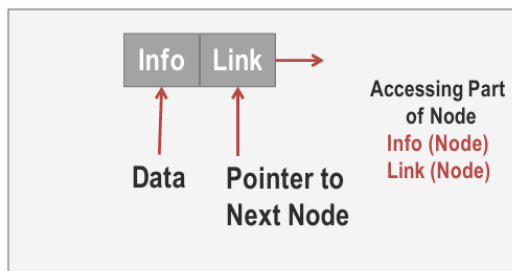
➤ Types of Link List

- Singly Linked List
- Circular Linked List
- Doubly Linked List

Singly Link List

➤ What is Singly Link List?

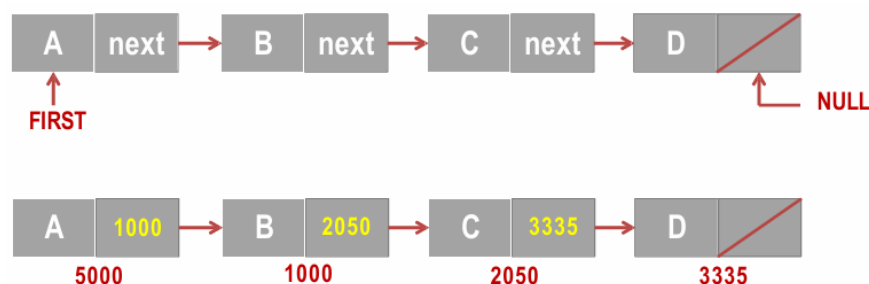
- It is basic type of linked list.
- Each node contains data and pointer to next node.
- Last node's pointer is null.
- First node address is available with pointer variable FIRST.
- Limitation of singly linked list is we can traverse only in one direction, forward direction



Typical Node

```
struct node
{
    int info;
    struct node *link;
};
```

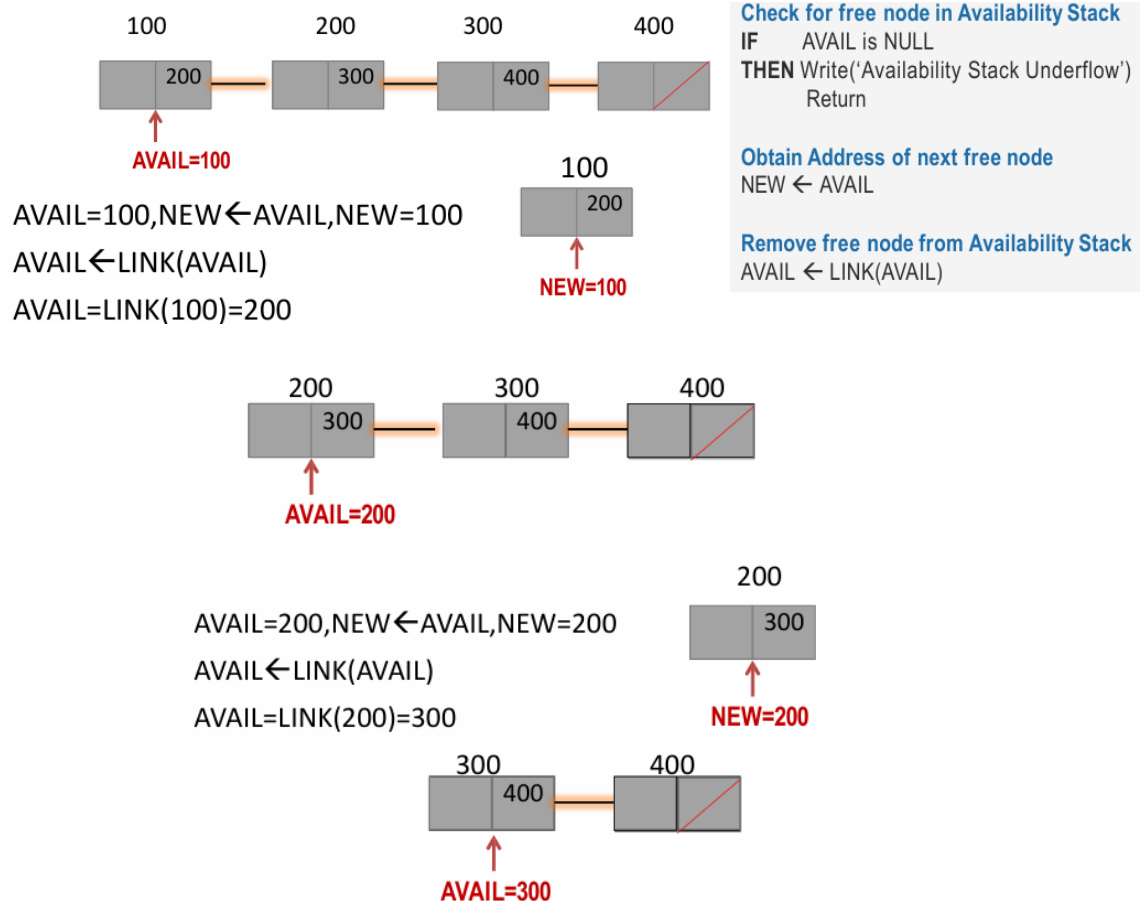
C Structure to represent a node



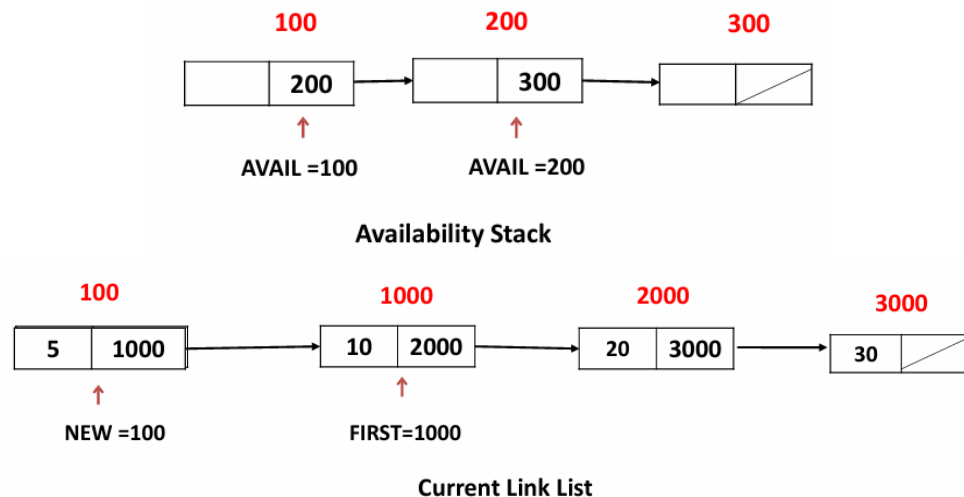
- Algorithms for singly linked list
 - Insert at first position
 - Insert at last position
 - Insert in Ordered Linked list
 - Delete Element
 - Copy Linked List

• Availability Stack

- A pool or list of free nodes, which we refer to as the availability stack is maintained in conjunction with linked allocation.
- Whenever a node is to be inserted in a list, a free node is taken from the availability stack and linked to the new list.
- On other end, the deleted node from the list is added to the availability stack.



➤ INSERT AT FIRST POSITION IN SINGLY LINKED LIST



○ INSERT (X, FIRST)

- Given X, a new element, and FIRST, a pointer to the first element of a linked linear list whose typical node contains INFO and LINK fields as previous described, this function inserts X. AVAIL is a pointer to the top element of the availability stack; NEW is a temporary pointer variable. It is required that X precede the node whose address is given by FIRST.

1. [Underflow?]

If AVAIL = NULL

then Write ('Availability Stack Underflow')

Return (FIRST)

2. [Obtain address of next free node]

NEW ← AVAIL

3. [Remove free node from availability stack]

AVAIL ← LINK (AVAIL)

4. [Initialize fields of new node and its link to the list]

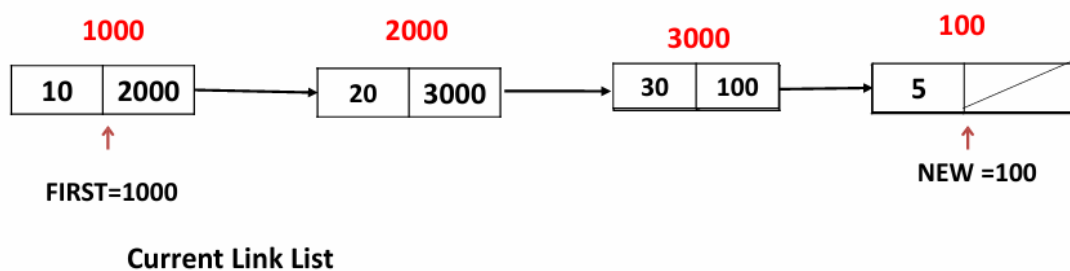
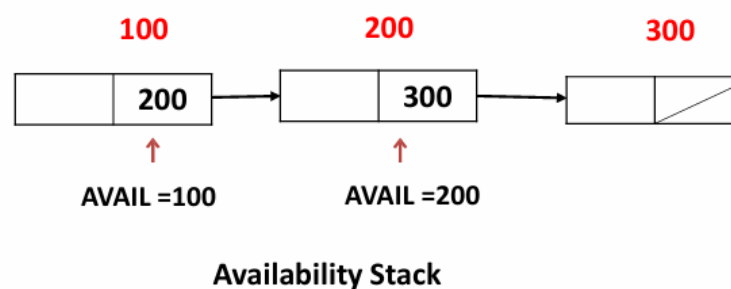
INFO (NEW) ← X

LINK (NEW) ← FIRST

5. [Return address of new node]

Return (NEW)

➤ INSERT AT LAST POSITION IN SINGLY LINK LIST



○ **INSEND (X, FIRST)**

- Given X, a new element, and FIRST, a pointer to the first element of a linked linear list whose typical node contains INFO and LINK fields as previously described, this function inserts X. AVAIL is a pointer to the top element of the availability stack; NEW and SAVE are temporary pointer variables. It is required that X be inserted at the end of the list.

1. [Underflow?]

If AVAIL = NULL

then Write ('Availability Stack Underflow')

Return (FIRST)

2. [Obtain address of next free node]

NEW ← AVAIL

3. [Remove free node from the availability stack]

AVAIL ← LINK (AVAIL)

4. [Initialize fields of new node]

INFO (NEW) ← X

LINK (NEW) ← NULL

5. [Is the list empty?]

If FIRST = NULL

then Return (NEW)

6. [Initiate search for the last node]

SAVE ← FIRST

7. [Search for end of list]

Repeat while LINK (SAVE) ≠ NULL

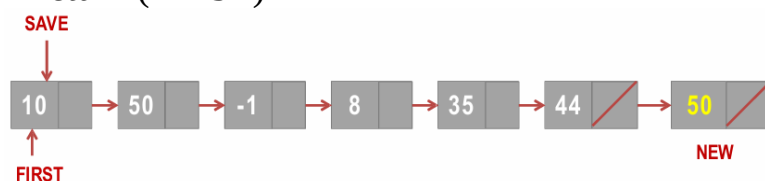
SAVE ← LINK (SAVE)

8. [Set LINK field of last node to NEW]

LINK (SAVE) ← NEW

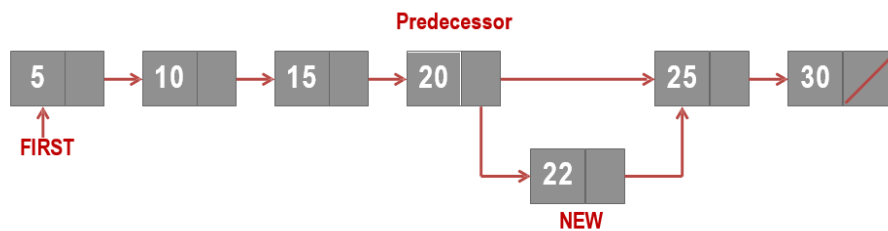
9. [Return first node pointer]

Return (FIRST)



➤ INSERT IN SORTED ORDER IN SINGLY LINK LIST

- **INSORD (X, FIRST)**
- Given a new term X, this function inserts it into a linked linear list whose typical node contains INFO and LINK fields as previously described. AVAIL is a pointer to the top element of the availability stack; NEW and SAVE are temporary pointer variables. It is required that X be inserted so that it preserves the ordering of the terms in increasing order of their INFO fields.



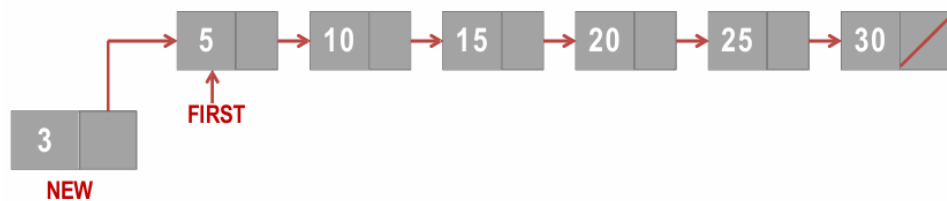
1. [Underflow?]
 - If AVAIL = NULL
 - then Write ('Availability Stack Underflow')
 - Return (FIRST)
2. [Obtain address of next free node]
 - NEW ← AVAIL
3. [Remove node from the availability stack]
 - AVAIL ← LINK (AVAIL)
4. [Copy information contents into new node]
 - INFO (NEW) ← X
5. [Is the list empty?]
 - If FIRST = NULL
 - then LINK (NEW) ← NULL
 - Return (NEW)
6. [Does the new node precede all others in the list?]
 - If INFO (NEW) ≤ INFO (FIRST)
 - then LINK (NEW) ← FIRST
 - Return (NEW)
7. [Initialize temporary pointer]
 - SAVE ← FIRST

8. [Search for predecessor of new node]
Repeat while $\text{LINK}(\text{SAVE}) \neq \text{NULL}$ and $\text{INFO}(\text{LINK}(\text{SAVE})) \leq \text{INFO}(\text{NEW})$
 $\text{SAVE} \leftarrow \text{LINK}(\text{SAVE})$
9. [Set link fields of new node and its predecessor]
 $\text{LINK}(\text{NEW}) \leftarrow \text{LINK}(\text{SAVE})$
 $\text{LINK}(\text{SAVE}) \leftarrow \text{NEW}$
10. [Return first node pointer]
Return (FIRST)

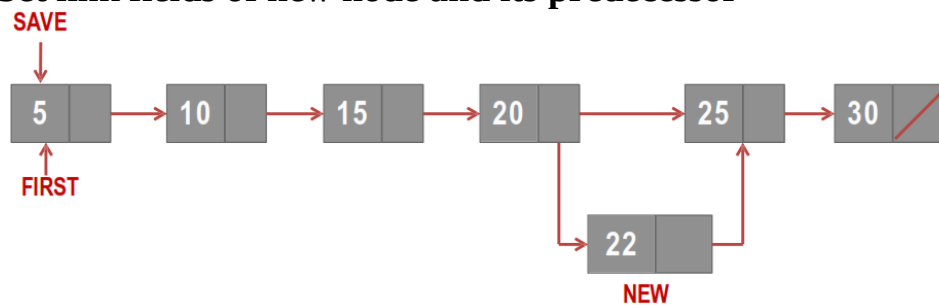
- Is the list empty?



- Does the new node precede all others in the list?



- Set link fields of new node and its predecessor

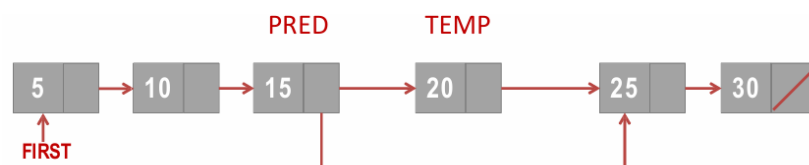


➤ DELETE OPERATION IN SINGLY LINK LIST

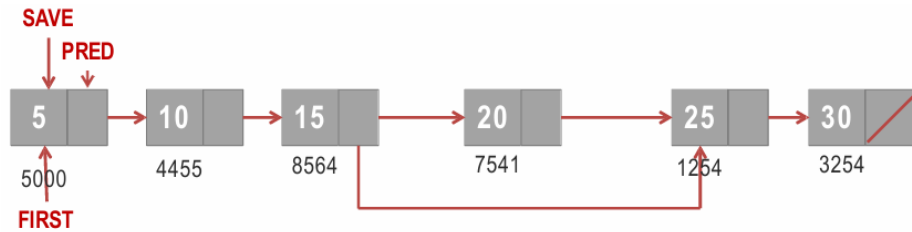
- DELETE (X, FIRST)
- Given X and FIRST, pointer variables whose values denote the address of a node in a linked list and the address of the first node in the linked list respectively, this procedure deletes the node whose address is given

by X. TEMP is used to find the desired node, and PRED keeps track of the predecessor of TEMP. Note that FIRST is changed only when X is the first element of the list.

1. [Empty list?]
If FIRST = NULL
then Write ('Underflow')
Return
2. [Initialize search of X]
TEMP $\leftarrow$ FIRST
3. [Find X]
Repeat thru step 5 while TEMP $\neq$ X and LINK (TEMP) $\neq$ NULL
4. [Update predecessor marker]
PRED $\leftarrow$ TEMP
5. [Move to next node]
TEMP $\leftarrow$ LINK (TEMP)
6. [End of list?]
If TEMP $\neq$ X
then Write ('Node not Found')
Return
7. [Delete X]
If X = FIRST (Is X the first node?)
then FIRST $\leftarrow$ LINK (FIRST)
else LINK (PRED) $\leftarrow$ LINK (X)
8. [Return node to availability area]
LINK (X) $\leftarrow$ AVAIL
AVAIL $\leftarrow$ X
Return



○ **DELETE(7541, FIRST)**



➤ COPY OPERATION IN SINGLY LINK LIST

○ COPY (FIRST)

- Given FIRST, a pointer to the first node in a linked list, this function makes a copy of this list. A typical node in the given list consists of INFO and LINK fields; the new list is to contain nodes whose information and pointer fields are denoted by FIELD and PTR respectively. The address of the first node in the newly created list is to be placed in BEGIN. NEW, SAVE and PRED are pointer variables.

1. [Empty list?]

If FIRST = NULL
then Return (NULL)

2. [Copy first node]

If AVAIL = NULL
then Write ('Availability Stack Underflow')
Return (0)
else NEW ← AVAIL
AVAIL ← LINK (AVAIL)
FIELD (NEW) ← INFO (FIRST)
BEGIN ← NEW

3. [Initialize traversal]

SAVE ← FIRST

4. [Move to next node if not at end of list]

Repeat thru step 6 while LINK (SAVE) ≠ NULL

5. [Update predecessor and save pointers]

PRED ← NEW
SAVE ← LINK (SAVE)

6. [Copy node]

If AVAIL = NULL

then Write ('Availability Stack Underflow')

Return (0)

else NEW $\leftarrow$ AVAIL

AVAIL $\leftarrow$ LINK (AVAIL)

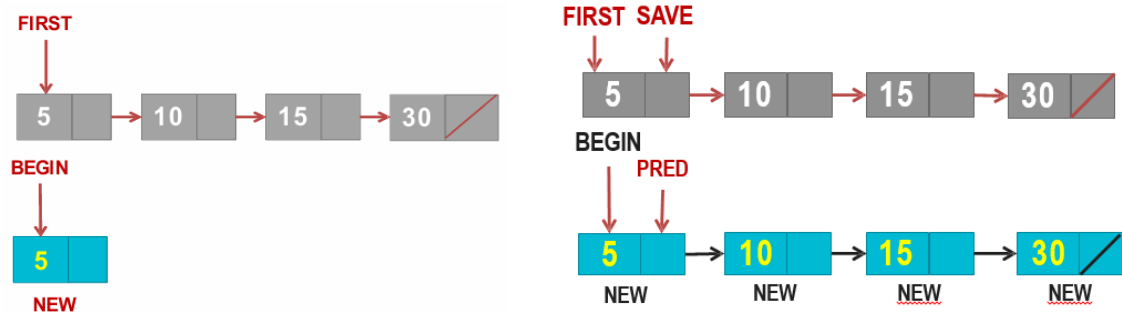
FIELD (NEW) $\leftarrow$ INFO (SAVE)

PTR (PRED) $\leftarrow$ NEW

7. [Set link of last node and return]

PTR (NEW) $\leftarrow$ NULL

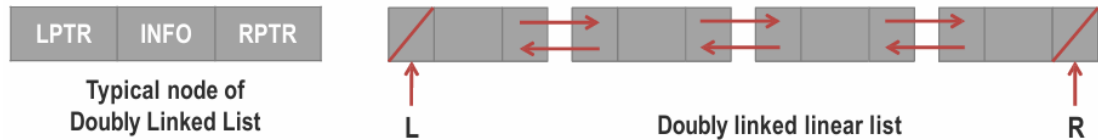
Return (BEGIN)



Doubly Link List

➤ Doubly Link List

- In certain Applications, it is very desirable that a list be traversed in either forward or reverse direction.
- This property implies that each node must contain two link fields instead of usual one.
- The links are used to denote Predecessor and Successor of node.
- The link denoting its predecessor is called Left Link.
- The link denoting its successor is called Right Link.
- A list containing this type of node is called doubly linked list or two way chain.
- Typical node of doubly linked linear list contains **INFO**, **LPTR**, **RPTR** Fields
- **LPTR** is pointer variable pointing to Predecessor of a node
- **RPTR** is pointer variable pointing to Successor of a node
- **Left most node** of doubly linked linear list is called **L**, **LPTR** of node **L** is always NULL
- **Right most node** of doubly linked linear list is called **R**, **RPTR** of node **R** is always NULL



• Algorithms for Doubly linked list

- Insert Operation in Doubly Link List
- Delete Operation in Doubly Link List

➤ INSERT OPERATION IN DOUBLY LINK LIST

- DOUBINS (L, R, M, X)
- Given a doubly linked linear list whose left-most and right-most node addresses are given by the pointer variables L and R, respectively, it is required to insert a node whose address is given by the pointer variable NEW. The left and right links of a node are denoted by LPTR and RPTR, respectively. The information field of a node is denoted by the variable INFO. The name of an element of the list is NODE. The insertion is to be performed to the left of a specified node with its address given by the pointer variable M. The information to be entered in the node is contained in X.

1. [Obtain new node from availability stack]
NEW $\leftarrow$ NODE
2. [Copy information field]
INFO (NEW) $\leftarrow$ X
3. [Insertion into an empty list?]
If R = NULL
then LPTR (NEW) $\leftarrow$ RPTR (NEW) $\leftarrow$ NULL
L $\leftarrow$ R $\leftarrow$ NEW
Return
4. [Left-most insertion?]
If M = L
then LPTR (NEW) $\leftarrow$ NULL
RPTR (NEW) $\leftarrow$ M
LPTR (M) $\leftarrow$ NEW
L $\leftarrow$ NEW
Return

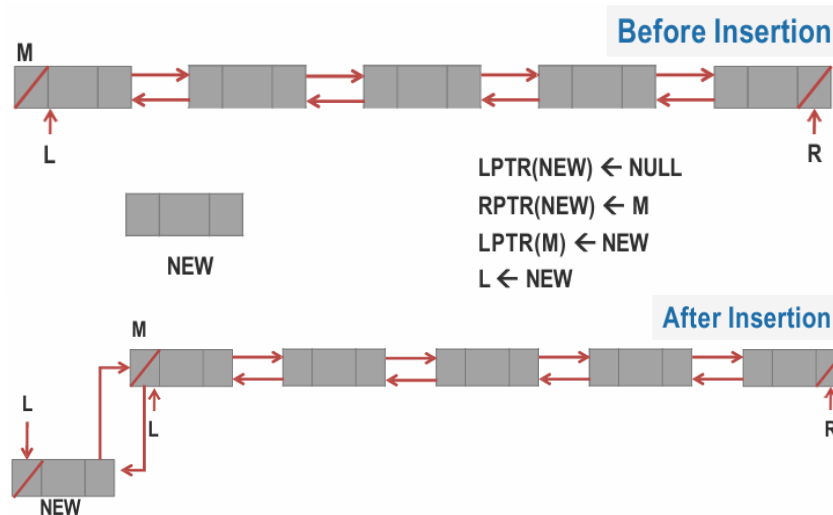
5. [Insert in middle]

```

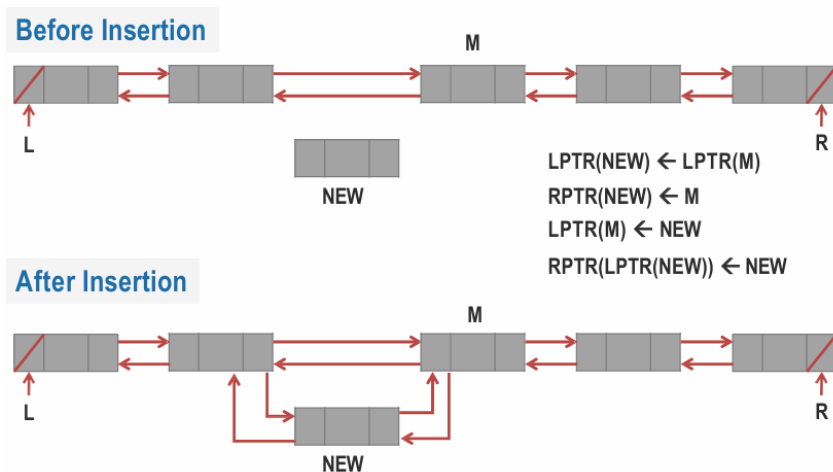
LPTR (NEW) ← LPTR (M)
RPTR (NEW) ← M
LPTR (M) ← NEW
RPTR (LPTR (NEW)) ← NEW
Return

```

○ Left most insertion in Doubly Linked Linear List



○ Insertion in the middle of Doubly Linked Linear List



➤ DELETE OPERATION IN DOUBLY LINK LIST

○ DOUBDEL (L, R, OLD)

- Given a doubly linked list with the addresses of the left-most and right-most nodes given by the pointer variables L and R, respectively, it is required to delete the node whose address is contained in the variable



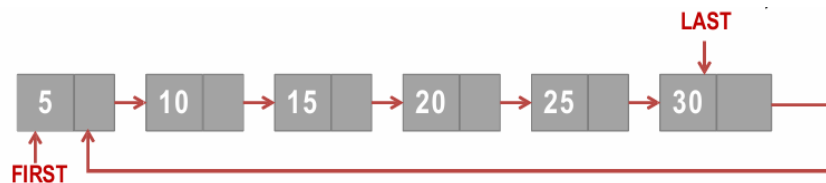
Return



- If we replace NULL pointer of the last node of Singly Linked Linear List with the address of its first node, that list becomes circularly linked linear list or Circular List.
- FIRST is the address of first node of Circular List
- LAST is the address of the last node of Circular List

- **Advantages of Circular List**

- In circular list, every node is accessible from given node
- It saves time when we have to go to the first node from the last node.
- It can be done in single step because there is no need to traverse the in between nodes.



- **Disadvantages of Circular List**

- It is not easy to reverse the linked list.
- If proper care is not taken, then the problem of infinite loop can occur.
- If we at a node and go back to the previous node, then we cannot do it in single step.
- Instead we have to complete the entire circle by going through the in between nodes and then we will reach the required node.

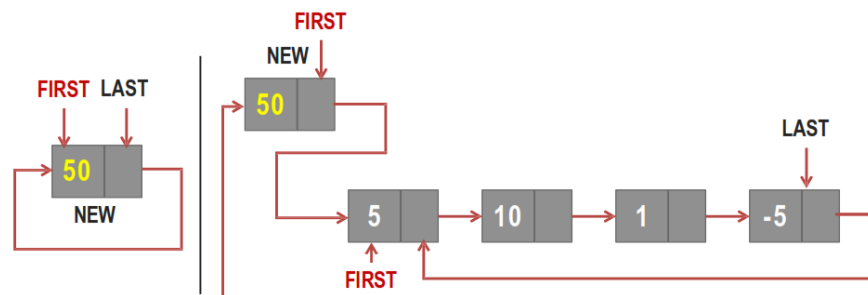
- **Operations on Circular List:**

- Insert at First
- Insert at Last
- Insert in Ordered List
- Delete a node

- **CIR_INS_FIRST(X,FIRST,LAST)**

- This procedure inserts a new node at the first position of Circular linked list.
- X is a new element to be inserted.
- FIRST and LAST are a pointer to the first & last elements of a Circular linked linear list, respectively.
- Typical node contains INFO and LINK fields.
- NEW is a temporary pointer variable

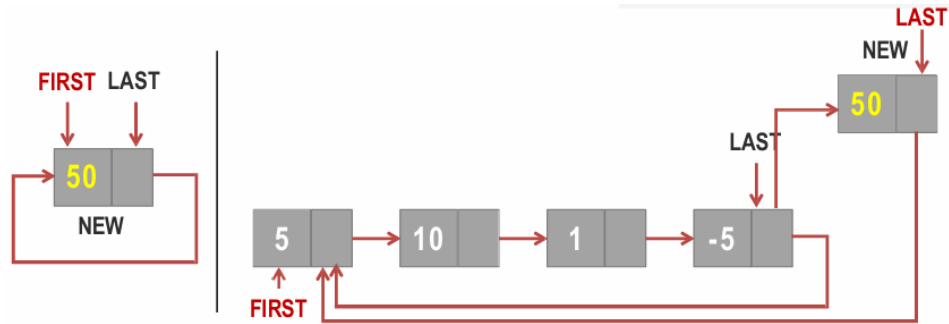
1. [Creates a new empty node]
NEW $\leftarrow$ NODE
2. [Initialize fields of new node and its link]
INFO (NEW) $\leftarrow$ X
IF FIRST = NULL
THEN LINK (NEW) $\leftarrow$ NEW
FIRST $\leftarrow$ LAST $\leftarrow$ NEW
ELSE LINK (NEW) $\leftarrow$ FIRST
LINK (LAST) $\leftarrow$ NEW
FIRST $\leftarrow$ NEW
Return



• CIR_INS_LAST(X,FIRST,LAST)

- This procedure inserts a new node at the Last position of Circular linked list.
- X is a new element to be inserted.
- FIRST and LAST are a pointer to the first & last elements of a Circular linked linear list, respectively.
- Typical node contains INFO and LINK fields.
- NEW is a temporary pointer variable.

1. [Creates a new empty node]
NEW $\leftarrow$ NODE
2. [Initialize fields of new node and its link]
INFO (NEW) $\leftarrow$ X
IF FIRST = NULL
THEN LINK (NEW) $\leftarrow$ NEW
FIRST $\leftarrow$ LAST $\leftarrow$ NEW
ELSE LINK (NEW) $\leftarrow$ FIRST
LINK (LAST) $\leftarrow$ NEW
LAST $\leftarrow$ NEW
Return

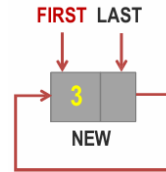


• **CIR_INS_ORD(X,FIRST,LAST)**

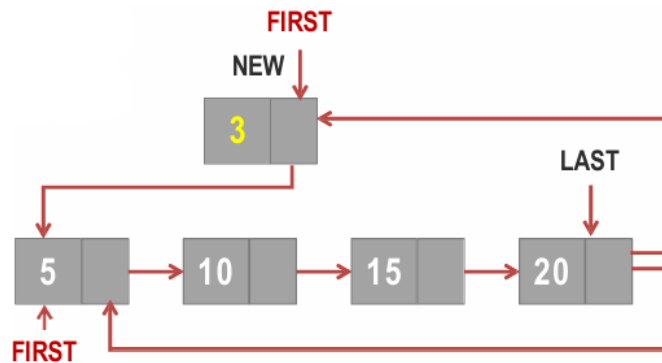
1. [Create New Empty Node]
NEW $\leftarrow$ NODE
2. [Copy information content into new node]
INFO(NEW) $\leftarrow$ X
3. [Is Linked List Empty?]
IF FIRST = NULL
THEN LINK(NEW) $\leftarrow$ NEW
FIRST $\leftarrow$ LAST $\leftarrow$ NEW
Return
4. [Does new node precedes all other nodes in List?]
IF INFO(NEW) $\leq$ INFO(FIRST)
THEN LINK(NEW) $\leftarrow$ FIRST
LINK(LAST) $\leftarrow$ NEW
FIRST $\leftarrow$ NEW
Return
5. [Initialize Temporary Pointer]
SAVE $\leftarrow$ FIRST
6. [Search for Predecessor of new node]
Repeat while SAVE $\neq$ LAST & INFO(NEW) $\geq$ INFO(LINK(SAVE))
SAVE $\leftarrow$ LINK(SAVE)
7. [Set link field of NEW node and its Predecessor]
LINK(NEW) $\leftarrow$ LINK(SAVE)
LINK(SAVE) $\leftarrow$ NEW
IF SAVE = LAST
THEN LAST $\leftarrow$ NEW

8. [Finished]
Return

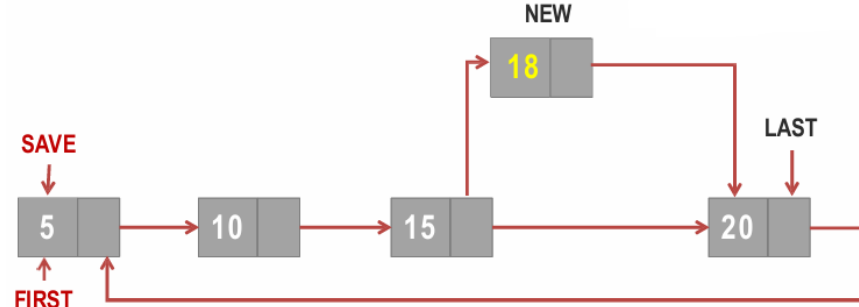
- Is Linked List Empty?



- Does new node precedes all other nodes in List?



- Set link field of NEW node and its Predecessor



• CIR_DELETE(X,FIRST,LAST)

- This algorithm delete a node whose address is given by variable X.
- FIRST & LAST are pointers to the First & Last elements of a Circular linked list, respectively.
- Typical node contains INFO and LINK fields.
- SAVE & PRED are temporary pointer variable.

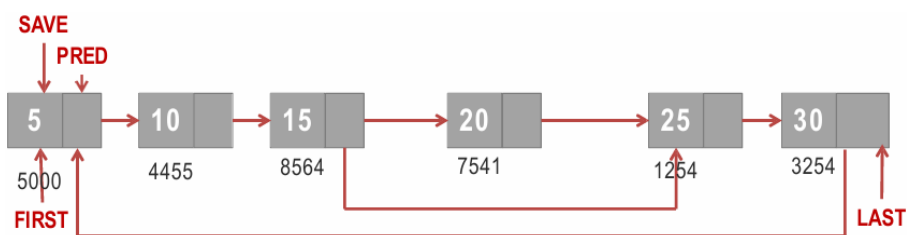
1. [Is Empty List?]

IF FIRST = NULL

THEN write('Linked List is Empty')

Return

2. [Initialize Search for X]
SAVE $\leftarrow$ FIRST
3. [Find X]
Repeat thru step5 while SAVE $\neq$ X & SAVE $\neq$ LAST
4. [Update predecessor marker]
PRED $\leftarrow$ SAVE
5. [Move to next node]
SAVE $\leftarrow$ LINK(SAVE)
6. [End of Linked List?]
IF SAVE $\neq$ X
THEN write('Node not found')
Return
7. [Delete X]
IF X = FIRST
THEN FIRST $\leftarrow$ LINK(FIRST)
LINK(LAST) $\leftarrow$ FIRST
ELSE LINK(PRED) $\leftarrow$ LINK(X)
IF X = LAST
THEN LAST $\leftarrow$ PRED
8. [Free Deleted Node]
Free (X)



Linked Representation of Stack

➤ Linked Representation of Stack

- We have seen how a stack is created using an array.
- This technique of creating a stack is easy, but the drawback is that the array must be declared to have some fixed size.

- In case the stack is a very small one or its maximum size is known in advance, then the array implementation of the stack gives an efficient implementation.
- But if the array size cannot be determined in advance, then the other alternative, i.e., linked representation, is used.
- The storage requirement of linked representation of the stack with n elements is $O(n)$, and the typical time requirement for the operations is $O(1)$.
- In a linked stack, every node has two parts—one that stores data and another that stores the address of the next node.
- The START pointer of the linked list is used as TOP.
- All insertions and deletions are done at the node pointed by TOP. If TOP = NULL, then it indicates that the stack is empty.
- The linked representation of a stack is shown in Fig.

• Push Operation

- The push operation is used to insert an element into the stack. The new element is added at the topmost position of the stack. Consider the linked stack shown in Fig.



- To insert an element with value 9, we first check if TOP=NULL. If this is the case, then we allocate memory for a new node, store the value in its DATA part and NULL in its NEXT part.
- The new node will then be called TOP. However, if TOP!=NULL, then we insert the new node at the beginning of the linked stack and name this new node as TOP. Thus, the updated stack becomes as shown in Fig.



- Figure shows the algorithm to push an element into a linked stack. In Step 1, memory is allocated for the new node.
- In Step 2, the DATA part of the new node is initialized with the value to be stored in the node.
- In Step 3, we check if the new node is the first node of the linked list.
- This is done by checking if TOP = NULL.
- In case the IF statement evaluates to true, then NULL is stored in the NEXT part of the node and the new node is called TOP. However, if the

new node is not the first node in the list, then it is added before the first node of the list (that is, the TOP node) and termed as TOP.

Step 1: Allocate memory for the new node and name it as NEW_NODE

Step 2: SET INFO(NEW_NODE) = VAL

Step 3: IF TOP = NULL

SET LINK(NEW_NODE) = NULL

SET TOP = NEW_NODE

ELSE

SET LINK(NEW_NODE) = TOP

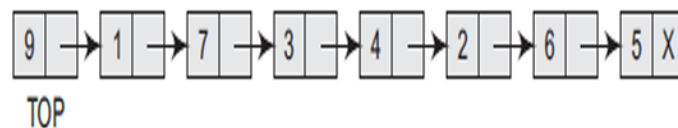
SET TOP = NEW_NODE

[END OF IF]

Step 4: END

• POP Operation

- The pop operation is used to delete the topmost element from a stack.
- However, before deleting the value, we must first check if TOP=NULL, because if this is the case, then it means that the stack is empty and no more deletions can be done.
- If an attempt is made to delete a value from a stack that is already empty, an UNDERFLOW message is printed. Consider the stack shown in Fig



- In case TOP!=NULL, then we will delete the node pointed by TOP, and make TOP point to the second element of the linked stack.
- Thus, the updated stack becomes as shown in Fig.



Step 1: IF TOP = NULL

PRINT "UNDERFLOW"

Go to Step 5

[END OF IF]

Step 2: SET PTR = TOP

Step 3: SET TOP = LINK(TOP)

Step 4: FREE PTR

Step 5: END

- Figure shows the algorithm to delete an element from a stack.
- In Step 1, we first check for the UNDERFLOW condition.
- In Step 2, we use a pointer PTR that points to TOP. In Step 3, TOP is made to point to the next node in sequence.
- In Step 4, the memory occupied by PTR is given back to the free pool.

Linked Representation of Queue

➤ Linked Representation of QUEUE

- We have seen how a queue is created using an array.
- Although this technique of creating a queue is easy, its drawback is that the array must be declared to have some fixed size.
- If we allocate space for 50 elements in the queue and it hardly uses 20–25 locations, then half of the space we have seen how a queue is created using an array.
- Although this technique of creating a queue is easy, its drawback is that the array must be declared to have some fixed size.
- If we allocate space for 50 elements in the queue and it hardly uses 20–25 locations, then half of the space will be wasted.
- And in case we allocate less memory locations for a queue that might end up growing large and large, then a lot of re-allocations will have to be done, thereby creating a lot of overhead and consuming a lot of time.
- **Insert Operation:**
 - The insert operation is used to insert an element into a queue. The new element is added as the last element of the queue.
 - Consider the linked queue shown in Fig

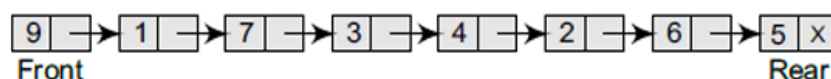


Figure 8.6 Linked queue

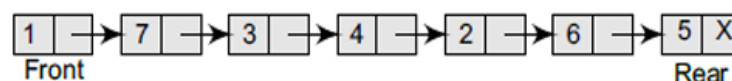
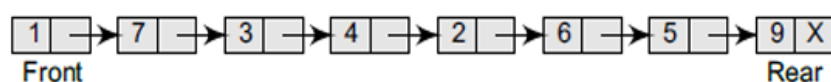


Figure 8.7 Linked queue



```

Step 1: Allocate memory for the new node and name it as PTR.
Step 2: SET INFO(PTR) = VAL
Step 3: IF FRONT = NULL
        SET FRONT = REAR = PTR
        SET LINK(FRONT) = LINK(REAR) = NULL
    ELSE
        SET LINK(REAR) = PTR
        SET REAR = PTR
        SET LINK(REAR) = NULL
[END OF IF]
Step 4: END
  
```

- Figure shows the algorithm to insert element in a linked queue.
- In Step 1, the memory is allocated for the new node.
- In Step2, the DATA part of the new node is initialized with the value to be stored in the node.
- In Step 3, we check if the new node is the first node of the linked queue.
- This is done by checking if FRONT = NULL.
- If this is the case, then the new node is tagged as FRONT as well as REAR.
- Also NULL is stored in the NEXT part of the node (which is also the FRONT and the REAR node).
- However, if the new node is not the first node in the list, then it is added at the REAR end of the linked queue (or the last node of the queue).

• Delete Operation:

- The delete operation is used to delete the element that is first inserted in a queue, i.e., the element whose address is stored in FRONT.
- However, before deleting the value, we must first check if FRONT=NULL because if this is the case, then the queue is empty and no more deletions can be done.
- If an attempt is made to delete a value from a queue that is already empty, an underflow message is printed. Consider the queue shown in Fig.

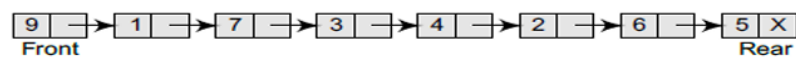
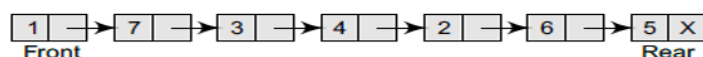


Figure 8.10 Linked queue



Step 1: IF FRONT = NULL
 Write "Underflow"
 Go to Step 5
 [END OF IF]
Step 2: SET PTR = FRONT
Step 3: SET FRONT = LINK(FRONT)
Step 4: FREE PTR
Step 5: END

- Figure shows the algorithm to delete an element from a linked queue.
- In Step 1, we first check for the underflow condition.
- If the condition is true, then an appropriate message is displayed, otherwise in Step 2, we use a pointer PTR that points to FRONT.
- In Step 3, FRONT is made to point to the next node in sequence.
- In Step 4, the memory occupied by PTR is given back to the free pool.

Applications of Linked List

➤ Applications:

- **Implementation of Stacks and Queues:** Stacks and queues are abstract data types that can be efficiently implemented using linked lists.
- **Dynamic Memory Allocation:** In systems programming, linked lists are frequently used to manage free memory blocks.
- **Graph Implementations:** Graphs are widely used in software development for modeling networks, relationships, and more. Linked lists are crucial for implementing adjacency lists, which represent graphs.
- **Real-Time Data Processing:** In real-time systems where data is continuously generated and processed, linked lists offer an efficient way to manage streams of data.
- **File Systems:** File systems, especially in operating systems, often use linked lists to manage files and directories.

Programs

➤ Operations on Singly Linked List:

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
```

```
    struct node *next;
};
struct node *head;

void begininsert ();
void lastinsert ();
void randominsert();
void begin_delete();
void last_delete();
void after_delete();
void display();
void search();
void main ()
{
    int choice =0;
    while(choice != 9)
    {
        printf("\n\n*****Main Menu*****\n");
        printf("\nChoose one option from the following list ...\n");

        printf("\n===== \n");
        printf("\n1.Insert in beginning\n2.Insert at last\n3.Insert a Node after given
index\n4.Delete from Beginning\n5.Delete from last\n6.Delete node after
specified location\n7.Show\n8.Exit\n");
        printf("\nEnter your choice?\n");
        scanf("\n%d",&choice);
        switch(choice)
        {
            case 1:
                begininsert();
                break;
            case 2:
                lastinsert();
                break;
            case 3:
                randominsert();
                break;
            case 4:
                begin_delete();
                break;
            case 5:
                last_delete();
                break;
```

```
        case 6:
            after_delete();
            break;
        case 7:
            display();
            break;
        case 8:
            exit(0);
            break;
        default:
            printf("Please enter valid choice..");
    }
}
}
void begininsert()
{
    struct node *ptr;
    int item;
    ptr = (struct node *) malloc(sizeof(struct node *));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter value\n");
        scanf("%d",&item);
        ptr->data = item;
        ptr->next = head;
        head = ptr;
        printf("\nNode inserted");
    }
}

void lastinsert()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
}
```

```
else
{
    printf("\nEnter value?\n");
    scanf("%d",&item);
    ptr->data = item;
    if(head == NULL)
    {
        ptr -> next = NULL;
        head = ptr;
        printf("\nNode inserted");
    }
    else
    {
        temp = head;
        while (temp -> next != NULL)
        {
            temp = temp -> next;
        }
        temp->next = ptr;
        ptr->next = NULL;
        printf("\nNode inserted");
    }
}
}

void randominsert()
{
    int i,loc,item;
    struct node *ptr, *temp;
    ptr = (struct node *) malloc (sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter element value");
        scanf("%d",&item);
        ptr->data = item;
        printf("\nEnter the location after which you want to insert ");
        scanf("\n%d",&loc);
        temp=head;
        for(i=0;i<loc;i++)
```

```
{
    temp = temp->next;
    if(temp == NULL)
    {
        printf("\ncan't insert\n");
        return;
    }

    ptr ->next = temp ->next;
    temp ->next = ptr;
    printf("\nNode inserted");
}
}
void begin_delete()
{
    struct node *ptr;
    if(head == NULL)
    {
        printf("\nList is empty\n");
    }
    else
    {
        ptr = head;
        head = ptr->next;
        free(ptr);
        printf("\nNode deleted from the beginning ...\n");
    }
}
void last_delete()
{
    struct node *ptr,*ptr1;
    if(head == NULL)
    {
        printf("\nlist is empty");
    }
    else if(head -> next == NULL)
    {
        head = NULL;
        free(head);
        printf("\nOnly node of the list deleted ...\n");
    }
    else
```

```
{
    ptr = head;
    while(ptr->next != NULL)
    {
        ptr1 = ptr;
        ptr = ptr ->next;
    }
    ptr1->next = NULL;
    free(ptr);
    printf("\nDeleted Node from the last ...\n");
}

void after_delete()
{
    struct node *ptr,*ptr1;
    int loc,i;
    printf("\n Enter the location of the node after which you want to perform
deletion \n");
    scanf("%d",&loc);
    ptr=head;
    for(i=0;i<loc;i++)
    {
        ptr1 = ptr;
        ptr = ptr->next;

        if(ptr == NULL)
        {
            printf("\nCan't delete");
            return;
        }
    }
    ptr1 ->next = ptr ->next;
    free(ptr);
    printf("\nDeleted node %d ",loc+1);
}

void display()
{
    struct node *ptr;
    ptr = head;
    if(ptr == NULL)
    {
        printf("Nothing to print");
    }
}
```

```
}  
else  
{  
    printf("\nprinting values . . . . \n");  
    while (ptr!=NULL)  
    {  
        printf("\n%d",ptr->data);  
        ptr = ptr -> next;  
    }  
}  
}
```

Output:

*****Main Menu*****

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

1

Enter value

10

Node inserted

*****Main Menu*****

Choose one option from the following list ...

=====



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 2: Linear Data Structure

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

2

Enter value?

30

Node inserted

*****Main Menu*****

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

3

Enter element value15

Enter the location after which you want to insert 1

Node inserted

*****Main Menu*****



Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

7

printing values

10

30

15

*****Main Menu*****

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

4

Node deleted from the beginning ...



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 2: Linear Data Structure

*****Main Menu*****

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

7

printing values

30

15

*****Main Menu*****

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

6

Enter the location of the node after which you want to perform deletion

1



Deleted node 2

*****Main Menu*****

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

7

printing values

30

*****Main Menu*****

Choose one option from the following list ...

=====

- 1.Insert in beginning
- 2.Insert at last
- 3.Insert a Node after given index
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Show
- 8.Exit

Enter your choice?

8

➤ **Implementation of Stack using Linked List:**

```
#include <stdio.h>
#include <stdlib.h>

// Structure to create a node with data and the next pointer
struct node {
    int info;
    struct node *ptr;
} *top, *top1, *temp;

int count = 0;
// Push() operation on a stack
void push(int data) {
    if (top == NULL)
    {
        top = (struct node *)malloc(1*sizeof(struct node));
        top->ptr = NULL;
        top->info = data;
    }
    else
    {
        temp = (struct node *)malloc(1*sizeof(struct node));
        temp->ptr = top;
        temp->info = data;
        top = temp;
    }
    count++;
    printf("Node is Inserted\n\n");
}

int pop() {
    top1 = top;

    if (top1 == NULL)
    {
        printf("\nStack Underflow\n");
        return -1;
    }
    else
    {
        top1 = top1->ptr;
        int popped = top->info;
        free(top);
    }
}
```

```
    top = top1;  
    count--;  
    return popped;  
}
```

```
void display() {  
    // Display the elements of the stack  
    top1 = top;  
  
    if (top1 == NULL)  
    {  
        printf("\nStack Underflow\n");  
        return;  
    }  
  
    printf("The stack is \n");  
    while (top1 != NULL)  
    {  
        printf("%d--->", top1->info);  
        top1 = top1->ptr;  
    }  
    printf("NULL\n\n");  
}
```

```
int main() {  
    int choice, value;  
    printf("\nImplementation of Stack using Linked List\n");  
    while (1) {  
        printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");  
        printf("\nEnter your choice : ");  
        scanf("%d", &choice);  
        switch (choice) {  
            case 1:  
                printf("\nEnter the value to insert: ");  
                scanf("%d", &value);  
                push(value);  
                break;  
            case 2:  
                printf("Popped element is :%d\n", pop());  
                break;  
            case 3:  
                display();  
                break;  
            case 4:  
                return 0;  
        }  
    }  
}
```

```
        break;
    case 4:
        exit(0);
        break;
    default:
        printf("\nWrong Choice\n");
    }
}
```

Output:

Implementation of Stack using Linked List

1. Push
2. Pop
3. Display
4. Exit

Enter your choice : 1

Enter the value to insert: 20
Node is Inserted

1. Push
2. Pop
3. Display
4. Exit

Enter your choice : 1

Enter the value to insert: 40
Node is Inserted

1. Push
2. Pop
3. Display
4. Exit

Enter your choice : 2
Popped element is :40

1. Push
2. Pop
3. Display
4. Exit

Enter your choice : 3
The stack is
20--->NULL

1. Push
2. Pop
3. Display
4. Exit

Enter your choice : 4

➤ **Implementation of Queue using Linked List:**

```
#include <stdio.h>
#include <stdlib.h>

// Structure to create a node with data and the next pointer
struct node {
    int data;
    struct node * next;
};

struct node * front = NULL;
struct node * rear = NULL;

// Enqueue() operation on a queue
void enqueue(int value) {
    struct node *ptr;
    ptr = (struct node * ) malloc(sizeof(struct node));
    ptr -> data = value;
    ptr -> next = NULL;
    if ((front == NULL) && (rear == NULL)) {
        front = rear = ptr;
    } else {
        rear -> next = ptr;
        rear = ptr;
    }
}
```

```
printf("Node is Inserted\n\n");
}

// Dequeue() operation on a queue
int dequeue() {
    if (front == NULL) {
        printf("\nUnderflow\n");
        return -1;
    } else {
        struct node * temp = front;
        int temp_data = front -> data;
        front = front -> next;
        free(temp);
        return temp_data;
    }
}

// Display all elements of the queue
void display() {
    struct node * temp;
    if ((front == NULL) && (rear == NULL)) {
        printf("\nQueue is Empty\n");
    } else {
        printf("The queue is \n");
        temp = front;
        while (temp) {
            printf("%d--->", temp -> data);
            temp = temp -> next;
        }
        printf("NULL\n\n");
    }
}

int main() {
    int choice, value;
    printf("\nImplementation of Queue using Linked List\n");
    while (choice != 4) {
        printf("1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
        printf("\nEnter your choice : ");
        scanf("%d", & choice);
        switch (choice) {
            case 1:
                printf("\nEnter the value to insert: ");
```

```
        scanf("%d", & value);
        enqueue(value);
        break;
    case 2:
        printf("Popped element is :%d\n", dequeue());
        break;
    case 3:
        display();
        break;
    case 4:
        exit(0);
        break;
    default:
        printf("\nWrong Choice\n");
    }
}
return 0;
}
```

Output:

Implementation of Queue using Linked List

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice : 1

Enter the value to insert: 10
Node is Inserted

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice : 1

Enter the value to insert: 20
Node is Inserted

- 1.Enqueue



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 2: Linear Data Structure

- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice : 1

Enter the value to insert: 30
Node is Inserted

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice : 2
Popped element is :10

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice : 3
The queue is
20--->30--->NULL

- 1.Enqueue
- 2.Dequeue
- 3.Display
- 4.Exit

Enter your choice : 4

GTU Questions from Unit 2

Sr. No	Questions	CO	Answer	Marks
1	What is linked list? States the advantages of linked list over array. Also list various types of the linked list. [NLJIET]	2	Pg 1,2	3
2	Write user defined 'C' function to insert node at a specific location in singly linked list.[NLJIET]	5	Pg 23,24, 25,26,27, 28,29	3
3	Write an algorithm to find length of a simple link list. [NLJIET] Steps to find the length of the Linked List: • Initialize count as 0. • Initialize a node pointer, curr = head. • Do following while curr is not NULL ○ curr = curr -> next ○ Increment count by 1. • Return count.	2	-	3
4	Write an algorithm to delete an item from the doubly linked list. [NLJIET]	2	Pg 12,13	3
5	Write user defined 'C' function to insert node at the end in circular linked list.[NLJIET]	5	Lab Manual	3
6	Compare Array and Link list. [NLJIET] Compare array and linked list. [NLJIET]	2	Pg 1	3
7	Enlist and explain any 3 applications of linked list. [NLJIET] List any three uses of linked list. [NLJIET] Outline and explain any 3 applications of linked list. [NLJIET]	2	Pg 23	3
8	List the advantages of doubly linked list over singly linked list.[NLJIET]	2	Pg 10	3
9	Write an algorithm for insertion of a new node at last position in Singly Linked List. [NLJIET]	2	Pg 5	4
10	Write an algorithm for deletion of a specific node from the Singly Linked List. [NLJIET]	2	Pg 7,8, 9	4
11	Design an algorithm to insert an element in doubly linked list. [NLJIET]	2	Pg 11,12	4
12	Design an algorithm to delete an element in doubly linked list. [NLJIET] Write an algorithm to delete next element from	2	Pg 12, 13	4



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 2: Linear Data Structure

	doubly linked list from given position. [NLJIET]			
13	Write user defined 'C' function to delete node from a specific location in doubly linked list. [NLJIET]	5	Lab Manual	4
14	Write an algorithm to insert a node in a Circular Link List at the FIRST position. [NLJIET]	2	Pg 14,15	4
15	Write user defined 'C' function to delete node from end in circular linked list. [NLJIET]	5	Lab Manual	4
16	Write an algorithm to implement queue using Linked List. [NLJIET]	2	Pg 21,22, 23	4
17	List and explain linked list applications. [NLJIET]	2	Pg 23	4
18	Write a 'C' program to implement queue using linked list. [NLJIET]	5	Pg 37,38, 39,40	7
19	Write an algorithm for INSERT operation to insert a node at a given position in a Link list. [NLJIET]	2	Pg 6,7	7
20	Write and explain algorithm for deletion in Singly Linked List. [NLJIET]	2	Pg 7,8, 9	7
21	Write and explain algorithm for insertion in Singly Linked List. [NLJIET]	2	Pg 6,7	7
22	Write an algorithm to insert an item as below: 1) At the start of the linked list 2) At the end of the linked list [NLJIET] Write an algorithm to implement singly linked list and its operation such as insert element at front, last and at any position. [NLJIET]	2	Pg 4,5	7
23	Write an algorithm to sort existing singly linked list in ascending order according to the information field. [NLJIET]	2	Pg 6,7	7
24	Explain creation, insertion and deletion of doubly linked list with example. [NLJIET] Outline creation, insertion and deletion of doubly linked list with example. [NLJIET]	2	Pg 10,11, 12,13	7
25	Write and Explain algorithm for insertion in doubly linked list. [NLJIET] What is doubly linked list? Write Algorithm for insertion and deletion in doubly linked list. [NLJIET] Outline doubly linked list? Create Algorithm for insertion and deletion in doubly linked list. [NLJIET]	2	Pg 11,12, 13	7



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 2: Linear Data Structure

26	Design an algorithm to merge two linked list. [NLJIET] Algorithm: If head1 is null, return head2. If head2 is null, return head1. Compare head1.data and head2.data. If head1.data <= head2.data: ○ set head = head1 ○ set head.next = merge(head1.next, head2) Else: ○ set head = head2 ○ set head.next = merge(head1, head2.next) Return head.	2	-	7
27	Write a 'C' program to implement stack using linked list. [NLJIET]	5	Pg 33,34, 35,36	7